

SINTAXIS Y SEMÁNTICA DE LOS LENGUAJES

TRABAJO PRÁCTICO INTEGRADOR:

DISEÑO E IMPLEMENTACIÓN

DE LEXER Y PARSER

INTÉRPRETE SMART-HOME Y TRADUCTOR A HTML



GRUPO: MULTIVAC

INTEGRANTES:

- CAVALLERO TOMAS
- DILAN MEISTER JULIÁN
- ESTIGARRIBIA JULIÁN GINES
- GONZALEZ LEANDRO AGUSTIN
- NUÑEZ LAUTARO JOSÉ
- ROMERO IVAN EZEQUIEL

PRIMER CUATRIMESTRE

CURSO ACADEMICO 2026

CHACO Y CORRIENTES, 12 DE ABRIL DEL 2026

Indice

Indice	2
Introducción	2
Conformación del Grupo	2
Alianza:	3
Gramatica	4
Descripción General	4
Símbolos de la gramática	4
2.1 Símbolos terminales	4
2.1 Símbolos no terminales	5
Reglas de Producción	7
Análisis Léxico	10
Descripción General	10
Explicación del código del lexer	10
Explicación visual	11
Análisis Sintáctico	12
Funciones auxiliares	15
1-Funciones auxiliares del analizador léxico (lexer.py)	15
2-Funciones auxiliares del analizador sintáctico (parser.py)	18
3. Clases auxiliares para el control de errores	23
4. Integración de las funciones auxiliares	23
Modo de Obtención del Intérprete	23
Modo de Ejecución del Intérprete	24
Conclusiones	25
Anexo	26

Introducción

Conformación del Grupo

Matriz de Habilidades

Habilidad x Integrante	Julian	Ivan	Tomas	Leandro	Dilan	Lautaro
Ingles	○	○	○	○	★	★
Buscar Info	★	★	○	★	○	○
Programar	○	○		○		★
Idear Soluciones	★	★	★	○	★	○
Escribir Documentación	○	★	○	★	○	★
Redes Sociales		○	○			○
Edición Video	○	○		○	○	★
Cebar Mates		○	○		★	★

Alianza:

En la correspondiente alianza confeccionada y aceptada por el grupo MULTIVAC, presenta:

- Responsabilidad en la realización de las partes asignadas por cada integrante del grupo Multivac, para la estructuración del TPI.
- Conectarse dos veces por semana a la plataforma “Discord y Whatsapp”, para la planificación y desarrollo del TPI.
- Uso de la aplicación Git para el desarrollo de los analizadores asincrónicos y la plataforma GitHub para el resguardo de progreso realizado.
- Comunicación de los posibles inconvenientes que cada integrante pueda tener en cuanto al progreso del trabajo y de las situaciones personales que puedan interferir con su realización, con un prudente tiempo de anticipación.

Gramatica

Descripcion General

SMART HOME es un lenguaje basado en eventos con ejecución reactiva. La gramática describe tres tipos de instrucciones de nivel superior: bloques **WHEN**

(disparados por eventos), bloques **EVERY** (disparados por tiempo) y sentencias condicionales o de asignación independientes.

Símbolos de la gramática

2.1 Símbolos terminales

Estos son los componentes léxicos atómicos que el parser recibe como entrada:

Token	Descripción	Ejemplos
when	Palabra reservada	when WHEN
every	Palabra reservada	every EVERY
if	Palabra reservada	if IF
then	Palabra reservada	then THEN
else	Palabra reservada	else ELSE
do	Palabra reservada	do DO
end	Palabra reservada	end END
and	Operador lógico	and AND
or	Operador lógico	or OR
not	Operador lógico	not NOT
igual	Operador de igualdad	==
distinto	Operador de diferencia	!=
mayor	Mayor que	>
menor	Menor que	<
mayor_igual	Mayor o igual	>=
menor_igual	Menor o igual	<=
asignacion	Asignación	=

punto	Acceso a atributo	.
sensor_luz_id	Identificador sensor de luz	sensor_luz
sensor_temp_id	Identificador sensor de temperatura	sensor_temp
sensor_humedad_id	Identificador sensor de humedad	sensor_humedad
sensor_movimiento_id	Identificador sensor de movimiento	sensor_movimiento

sensor_humo_id	Identificador sensor de humo	sensor_humo
foco_id	Identificador de foco	Foco_entrada, foco_sala
aire_id	Identificador de aire	aire_salon, aire_habitacion
persiana_id	Identificador de persiana	persiana_sala
cerradura_id	Identificador de cerradura	cerradura_principal
reloj_id	Identificador de reloj	reloj_salon
altavoz_id	Identificador de altavoz	altavoz_comedor
alarma_id	Identificador de alarma	alarma_casa
bool_sensor	Valor booleano de sensor	TRUE, FALSE
bool_dispositivo	Valor booleano de dispositivo	ON, OFF
temp_val	Literal de temperatura	25°C -5°C
porcentaje_val	Literal de porcentaje	80% 100%
tiempo_val	Literal de tiempo/duración	30m 10s 1h
lux_val	Literal de iluminancia	250lux 600lux
hora_val	Literal de hora	22:00 06:30
fecha_val	Literal de fecha	21/04/2026
text_val	Cadena de texto entre comillas	“Hola mundo”
email_val	Dirección de correo electrónico	user@dominio.com
discreto_val	Valor discreto de dispositivo	FRIO CALOR VENT
color_val	Valor de color	blanco rojo azul
comentario	Comentario de línea (ignorado)	// texto

2.1 Símbolos no terminales

Estos son los constructores sintácticos de la gramática:

No terminal (mayusculas)	Descripción
PROGRAMA	Símbolo inicial, secuencia de instrucciones
LISTA_SENTENCIAS	Agrupación de acciones
SENTENCIA	Unidad mínima de acción
BLOQUE_WHEN	Bloque disparado por evento

BLOQUE_EVERY	Bloque disparado por tiempo periódico
CONDICIONAL	Estructura IF ... THEN ... ELSE
LISTA_ACCIONES	Agrupación de acciones
ACCION	Unidad mínima de accion
ASIGNACION	Agrupador de asignaciones por tipo de dispositivo
ASIGNACION_FOCO	Sentencia de asignación para dispositivo foco.
ASIGNACION_AIRE	Sentencia de asignación para dispositivo aire acondicionado.
ASIGNACION_PERSIANA	Sentencia de asignación para dispositivo persiana.
ASIGNACION_CERRADURA	Sentencia de asignación para dispositivo cerradura.
ASIGNACION_ALTAVOZ	Sentencia de asignación para dispositivo altavoz.
ASIGNACION_ALARMA	Sentencia de asignación para dispositivo alarma.
ATRIB_FOCO	Atributo y valor válidos para foco
ATRIB_AIRE	Atributo y valor válido para aire .
ATRIB_PERSIANA	Atributo y valor válido para persiana
ATRIB_CERRADURA	Atributo y valor válido para persiana
ATRIB_ALTAVOZ	Atributo y valor válido para altavoz
ATRIB_ALARMA	Atributo y valor válido para alarma
CONDICION	Expresión booleana con or
CONDICION_AND	Expresión booleana con AND
CONDICION_NOT	Expresión booleana con NOT
EXPRESIÓN	Comparación completa entre operando y valor

COMPARACIÓN	Expresión relacional atómica
OPERADOR	Operador relacional
TIEMPO	Valores de tiempo
VALOR_BLSEN	Cualquier literal valido del lenguaje
VALOR_TEMP	Temperatura (-10.0 a 50.0 °c)
VALOR_HUM	Nivel de humedad ambiental(0% a 100%)
VALOR_LUX	Nivel de luz (0 a 1000)

Reglas de Producción

Los símbolos terminales estarán en minúsculas mientras que los no terminales en mayúsculas.

1.º Estructura del programa

$\Sigma \rightarrow \text{PROGRAMA}$

$\text{PROGRAMA} \rightarrow \text{LISTA_SENTENCIAS}$

$\text{LISTA_SENTENCIAS} \rightarrow \text{SENTENCIA}$

$\quad \quad \quad | \text{SENTENCIA LISTA_SENTENCIAS}$

$\text{SENTENCIA} \rightarrow \text{BLOQUE_WHEN}$

$\quad \quad \quad | \text{BLOQUE_EVERY}$

$\quad \quad \quad | \text{CONDICIONAL}$

$\quad \quad \quad | \text{ASIGNACION}$

2.º Bloques de control

$\text{BLOQUE_WHEN} \rightarrow \text{when CONDICION do LISTA_ACCIONES end}$

$\text{BLOQUE_EVERY} \rightarrow \text{every TIEMPO do LISTA_ACCIONES end}$

$\text{TIEMPO} \rightarrow \text{tiempo_val}$

3.º Condicional

$\text{CONDICIONAL} \rightarrow \text{if CONDICION then LISTA_ACCIONES end}$

$\quad \quad \quad | \text{if CONDICION then LISTA_ACCIONES else LISTA_ACCIONES end}$

4.º Acciones

$\text{LISTA_ACCIONES} \rightarrow \text{ACCION}$

| ACCION LISTA_ACCIONES

ACCION → ASIGNACION
| CONDICIONAL

5.º ASIGNACIONES - agrupador

ASIGNACION → ASIGNACION_FOCO
| ASIGNACION_AIRE
| ASIGNACION_PERSIANA
| ASIGNACION_CERRADURA
| ASIGNACION_ALTAVOZ
| ASIGNACION_ALARMA

6.º ASIGNACIONES POR DISPOSITIVO

ASIGNACION_FOCO → foco_id punto ATRIB_FOCO
ASIGNACION_AIRE → aire_id punto ATRIB_AIRE
ASIGNACION_PERSIANA → persiana_id punto ATRIB_PERSIANA
ASIGNACION_CERRADURA → cerradura_id punto ATRIB_CERRADURA
ASIGNACION_ALTAVOZ → altavoz_id punto ATRIB_ALTAVOZ
ASIGNACION_ALARMA → alarma_id punto ATRIB_ALARMA

7.º ASIGNACIONES ATRIBUTO

ATRIB_FOCO → estado asignacion bool_dispositivo
| brillo asignacion porcentaje_val
| color asignacion color_val

ATRIB_AIRE → estado asignacion bool_dispositivo
| modo asignacion discreto_val
| temp_obj asignación temp_val

ATRIB_PERSIANA → posicion asignación porcentaje_val

ATRIB_CERRADURA → estado asignación bool_dispositivo
|

ATRIB_ALTAVOZ → volumen asignacion porcentaje_val
| mute asignacion bool_dispositivo
| mensaje asignacion text_val
| email_notif asignacion email_val

ATRIB_ALARMA → estado asignacion bool_dispositivo

| activada asignacion bool_dispositivo

8.ºCONDICIONES

CONDICION → CONDICION_AND
| CONDICION_AND or CONDICION

CONDICION_AND → CONDICION_NOT
| CONDICION_NOT and CONDICION_AND

CONDICION_NOT → not CONDICION_NOT
| EXPRESION

9º. EXPRESIONES - Específicas por tipo de operando

EXPRESION → sensor_humo_id OPERADOR bool_sensor
| sensor_movimiento_id OPERADOR bool_sensor
| sensor_luz_id OPERADOR lux_val
| sensor_humedad_id OPERADOR porcentaje_val
| sensor_temp_id OPERADOR temp_val
| foco_id punto estado OPERADOR bool_dispositivo
| foco_id punto brillo OPERADOR porcentaje_val
| aire_id punto estado OPERADOR bool_dispositivo
| aire_id punto modo OPERADOR discreto_val
| aire_id punto temp_obj OPERADOR temp_val
| aire_id punto temp_act OPERADOR temp_val
| persiana_id punto posicion OPERADOR porcentaje_val
| cerradura_id punto estado OPERADOR bool_dispositivo
| reloj_id punto hora OPERADOR hora_val
| reloj_id punto fecha OPERADOR fecha_val
| altavoz_id punto volumen OPERADOR porcentaje_val
| altavoz_id punto mute OPERADOR bool_dispositivo
| altavoz_id punto mensaje OPERADOR text_val
| altavoz_id punto email_notif OPERADOR email_val
| alarma_id punto estado OPERADOR bool_dispositivo
| alarma_id punto activada OPERADOR bool_dispositivo

10º. OPERADORES

OPERADOR → igual | distinto | mayor | menor | mayor_igual | menor_igual

11º. VALORES DE SENSOR
VALOR_BLSEN→ bool_sensor
VALOR_TEMP→ temperatura_val
VALOR_HUM → porcentaje_val
VALOR_LUX → lux_val

Analisis Léxico

Descripcion General

Para la identificación de los tokens del lenguaje de SMART HOME utilizaremos un lexer realizado en python.

Explicacion del codigo del lexer

El lexer para el lenguaje "SMART HOME" describe reglas para dispositivos del hogar (focos, aires, persianas, alarmas, etc.). Su trabajo es tomar el texto crudo que escribe el usuario y convertirlo en una lista de tokens (unidades con significado: números, palabras reservadas, fechas, horas, emails, etc.), reportando errores léxicos cuando algo no encaja con ninguna regla válida.

Está construido como un autómata finito determinístico (AFD) hecho a mano, no con expresiones regulares ni generadores de lexers.

Primero se definen “las palabras del lenguaje” PALABRAS_RESERVADAS, BOOLEANOS_DISPOSITIVO y separado BOOLEANOS_SENSOR, OPERADORES_VALIDOS; ATRIBUTOS_POR_DISPOSITIVO (diccionario que dice qué atributos son válidos para cada dispositivo, ej. foco solo admite estado, brillo, color); DISPOSITIVOS_VALIDOS, SENSORES_VALIDOS; y tablas auxiliares para validar fechas reales (DIAS_POR_MES, NOMBRE_MES). Luego se definen los tipos de tokens(tipo, valor, línea, columna) y de errores lexicos (mensaje, línea, columna). Además de las funciones es_digito, es_letra, es_solo_digitos reemplazan a isdigit()/isalpha() del propio python.

La función Clasificar_caracter — Es el primer filtro: toma un carácter y lo traduce a una categoría genérica (no todavía un token): %→Porcentaje. Esto reduce la cantidad de caracteres a controlar haciendo manejable la tabla del autómata, siendo que la misma es el corazón del funcionamiento del autómata la cual define a qué estado pasa este con cada carácter. Cabe aclarar que el lenguaje no contempla operadores aritméticos, por lo que el carácter "+" no se clasifica como tal, sino únicamente como parte válida de la escritura de un correo electrónico (email_val),

donde se lo trata como un carácter más dentro del token.

ej (INICIO→/→BARRA_INICIAL→/→COMENTARIO), siendo el estado ACEPTACIÓN el que separa un token de otro .

Cuando vamos al apartado del bucle observamos que se recorre el string caracter por caracter con estado_actual, un acumulador y contadores de línea/columna. Por cada carácter lo clasifica y busca la transición: si es ACEPTACIÓN cierra el token y reprocesa el mismo carácter (sin avanzar el índice); si es normal, acumula y avanza de estado; si lleva a ERROR o no hay transición definida, genera un ErrorLexico

Además el lexer se encarga del control semántico de los tokens y como se fueron generando con el fin de evitar que se genere por ejemplo asignarle al volumen de un altavoz un color, usando una validación de si el contenido del token es correcto

Explicación visual

Ejemplo de Token correctos:

```
if foco.estado == on then
```

```
    aire.temp_obj = 25°c
```

```
End
```

Salida del lexer:

1	Token(RESERVADA, 'if', linea=1, col=1)
2	Token(DISPOSITIVO, 'foco', linea=1, col=4)
3	Token(PUNTO, '.', linea=1, col=8)
4	Token(ATRIBUTO, 'estado', linea=1, col=9)
5	Token(OPERADOR, '==', linea=1, col=16)
6	Token(BOOLEANO_DISPOSITIVO, 'on', linea=1, col=19)
7	Token(RESERVADA, 'then', linea=1, col=22)
8	Token(DISPOSITIVO, 'aire', linea=2, col=5)
9	Token(PUNTO, '.', linea=2, col=9)
10	Token(ATRIBUTO, 'temp_obj', linea=2, col=10)
11	Token(OPERADOR, '=', linea=2, col=19)
12	Token(TEMPERATURA, '25°c', linea=2, col=21)
13	Token(RESERVADA, 'end', linea=3, col=1)

En este caso todos los token cargadores son correctos con lo cual la salida dictara cuales son cada uno de los token (tipo) que se utilizaron y en qué sitio fue aceptado cada uno.

String de ejemplo (con tokens erroneos)

```
If sensor_humo_id = true then alar@ma_id.activada = OoN
```

Salida del lexer:

#	Token
1	Token(RESERVADA, 'If', linea=1, col=1)
2	Token(SENSOR, 'sensor_humo_id', linea=1, col=4)
3	Token(OPERADOR, '=', linea=1, col=19)
4	Token(BOOLEANO_SENSOR, 'true', linea=1, col=21)
5	Token(RESERVADA, 'then', linea=1, col=26)
6	Token(OPERADOR, '=', linea=1, col=51)

- Error léxico en línea 1, columna 45: Carácter 'i' no esperado en contexto (estado EXTENSION4)
- Error léxico en línea 1, columna 46: Identificador desconocido 'vada'
- Error léxico en línea 1, columna 53: Identificador desconocido 'OoN'

String con tokens aleatorios:

Vigil123@gmail.ar when 125% False 12/07/1999 On
Salida del lexer:

- 1 Token(EMAIL, 'Vigil123@gmail.ar', linea=1, col=1)
- 2 Token(RESERVADA, 'when', linea=1, col=19)
- 3 Token(BOOLEANO_SENSOR, 'False', linea=1, col=30)
- 4 Token(FECHA, '12/07/1999', linea=1, col=36)
- 5 Token(BOOLEANO_DISPOSITIVO, 'On', linea=1, col=47)

En este caso el lexer no detectó ningún error con lo cual no hay una salida no deseable.

Analisis Sintactico

El módulo parser.py implementa la etapa de análisis sintáctico del compilador, encargada de verificar que la secuencia de tokens producida por el analizador léxico respete la gramática definida para el lenguaje Smart Home. Se optó por una implementación de tipo recursivo descendente, en la cual cada regla de la gramática se traduce directamente en una función que reconoce esa estructura, consumiendo tokens de izquierda a derecha sin retroceder.

Esta implementación realiza una validación directa: si la secuencia de tokens es válida según la gramática, el proceso finaliza sin errores; si en algún punto un token no corresponde con lo esperado, se interrumpe la ejecución mediante una excepción que informa la posición exacta (línea y columna) y el tipo de token que se esperaba encontrar.

La función consumir

Es sobre la que se apoya todo el análisis. Recibe como parámetros el tipo de token esperado y, opcionalmente, un valor literal esperado (utilizado para palabras reservadas u operadores fijos, como `==`). Su funcionamiento consiste en tres validaciones sucesivas:

1. Verifica que existan tokens restantes por procesar.
2. Verifica que el tipo del token actual coincida con el tipo esperado.
3. En caso de haberse especificado un valor, verifica que dicho valor coincida exactamente.

Si las tres condiciones se cumplen, la función avanza el puntero de posición (`pos`) y retorna el token consumido. En caso contrario, lanza una excepción de error de sintaxis con el detalle correspondiente. Esta función centraliza todo el manejo de errores sintácticos del parser.

La función `ErrorSintactico`

Antes de entrar en las funciones de parseo propiamente dichas, el módulo define su propia excepción, `ErrorSintactico`, que además del mensaje guarda línea, columna, una sugerencia opcional y el contexto: una lista de nombres de producción (por ejemplo `["PROGRAMA", "BLOQUE_WHEN", "LISTA_ACCIONES", "ASIGNACION"]`) que indica en qué parte del árbol gramatical se produjo el error.

Ese contexto se arma con el gestor `contexto(nombre)`: cada función de parseo lo usa con `with contexto("NOMBRE_PRODUCCION")`: al entrar, lo que apila el nombre en `_pila_contexto`, y automáticamente lo desapila al salir del bloque (haya o no error). Así, si algo falla cinco niveles de recursión más abajo, el mensaje final puede reconstruir el camino completo en vez de dar sólo línea y columna.

Punto de entrada: `parsear(tokens)`

Es la función que invoca el resto del compilador. Hace tres cosas:

- A. Filtra los tokens de tipo `COMENTARIO` (la gramática dice que se ignoran, así que se descartan acá una sola vez en vez de que cada función de parseo tenga que esquivarlos).
- B. Inicializa el estado global (`pos = 0`, limpia `_pila_contexto`) y llama a `parse_programa()`, que es el símbolo inicial de la gramática.
- C. Envuelve la llamada en un `try/except` doble: si salta `ErrorSintactico`, imprime el detalle y devuelve `False` (cadena rechazada); si salta cualquier otra excepción no prevista, la trata como bug del parser (no del programa de entrada) y también devuelve `False`.

Funciones que representan reglas gramaticales

1. parse_programa
2. parse_lista_sentencias
3. parse_sentencia
4. parse_bloque_when
5. parse_bloque_every
6. parse_condicional
7. parse_lista_acciones
8. parse_accion
9. parse_condicion
10. parse_condicion_and
11. parse_condicion_not
12. parse_expresion
13. _parse_expresion_sensor
14. _parse_expresion_dispositivo
15. parse_asignacion

Funciones auxiliares

El intérprete Smart Home se apoya en un conjunto de funciones auxiliares que abstraen tareas repetitivas, facilitan el mantenimiento del código y centralizan la lógica de validación y control de errores. Estas funciones se distribuyen en los módulos `lexer.py` y `parser.py`, y se describen a continuación.

1-Funciones auxiliares del analizador léxico (`lexer.py`)

1.1. `es_digito(c)`, `es_letra(c)`, `es_solo_digitos(texto)`

```
def es_digito(c):  
    return '0' <= c <= '9'  
  
def es_letra(c):  
    return ('a' <= c <= 'z') or ('A' <= c <= 'Z')  
  
def es_solo_digitos(texto):  
    if len(texto) == 0:  
        return False  
    for c in texto:  
        if not es_digito(c):  
            return False  
    return True
```

Propósito:

Reemplazan a los métodos nativos de Python `isdigit()` y `isalpha()` para evitar dependencias de la configuración regional y garantizar un comportamiento determinista. Se utilizan en la clasificación de caracteres y en la validación de tokens numéricos y alfabéticos.

Implementación:

- `es_digito(c)`: retorna True si c está entre '0' y '9'.
- `es_letra(c)`: retorna True si c es una letra mayúscula o minúscula del alfabeto inglés.
- `es_solo_digitos(texto)`: recorre el string y verifica que todos sus caracteres sean dígitos.

Uso:

Se emplean en `clasificar_caracter()` para categorizar cada carácter, y en la validación de fechas, horas, porcentajes, etc.

1.2. `clasificar_caracter(caracter)`

```
def clasificar_caracter(caracter):  
    match caracter:  
        case '%': return 'Porcentaje'  
        case '°': return 'Grados'  
        case ':': return 'Separador_horario'  
        case '/': return 'Barra'  
        case '-': return 'Menos'  
        case '+': return 'Mas'  
        case '.': return 'Selector'  
        case '_': return 'Guion'  
        case '@': return 'Arroba'  
        case '"': return 'Comillas'  
        case '\n': return 'NuevaLinea'  
        case ' ': return 'Espacio'  
        case '\t': return 'Espacio'  
        case '\r': return 'Espacio'  
        case _ if caracter in OPERADORES_BASICOS: return 'Operador'  
        case _ if es_digito(caracter): return 'Numero'  
        case _ if es_letra(caracter): return 'Letra'  
        case _: return 'Desconocido'
```

Propósito:

Traduce cada carácter de la entrada a una categoría semántica (p. ej., 'Letra', 'Numero', 'Operador', 'Espacio', 'Barra', etc.). Esta clasificación permite que la tabla de transiciones del autómata finito esté definida en términos de categorías, reduciendo drásticamente el número de estados y transiciones necesarias.

Implementación:

Utiliza una estructura match-case que asigna a cada carácter una etiqueta. Por ejemplo, el carácter '%' se clasifica como 'Porcentaje', '/' como 'Barra', los dígitos como 'Numero', etc. Los caracteres no contemplados se etiquetan como 'Desconocido', lo que dispara un error léxico.

Uso:

Es invocada en cada iteración del bucle principal de `motor_lexer()` para obtener la categoría del carácter actual y consultar la tabla de transiciones.

1.3. `_validar_fecha_con_dia_mes(acumulador)`

```
def _validar_fecha_con_dia_mes(acumulador):
    partes = acumulador.split('/')
    if len(partes) != 3:
        return f"Formato inválido '{acumulador}': se esperaba DD/MM/AAAA"

    dia_str, mes_str, anio_str = partes
    if not es_solo_digitos(dia_str):
        return f"Día inválido '{dia_str}': debe ser un número"
    if not es_solo_digitos(mes_str):
        return f"Mes inválido '{mes_str}': debe ser un número"
    if not es_solo_digitos(anio_str):
        return f"Año inválido '{anio_str}': debe ser un número"

    dia = int(dia_str)
    mes = int(mes_str)
    anio = int(anio_str)
    #sigue...
```

Propósito:

Valida que una cadena con formato DD/MM/AAAA sea una fecha válida según las restricciones del lenguaje (día entre 1 y 31, mes entre 1 y 12, año entre 1900 y 2099). Además, verifica que el día no exceda el máximo de días del mes correspondiente (sin considerar años bisiestos, según la consigna).

Implementación:

Divide la cadena por '/', convierte cada parte a entero y aplica las comprobaciones. Si encuentra algún error, retorna un mensaje descriptivo; en caso contrario, retorna None.

Uso:

Se invoca desde `aceptar_token()` cuando el estado actual es `FECHA_ANIO` o `FECHA_MES`, justo antes de crear el token `FECHA`.

1.4. `_validar_hora(acumulador)`

```
def _validar_hora(acumulador):
    partes = acumulador.split(':')
    if len(partes) != 2:
        return f"Formato inválido '{acumulador}': se esperaba HH:MM"

    hora_str, min_str = partes
    if len(hora_str) != 2 or not es_solo_digitos(hora_str):
        return f"Hora inválida '{hora_str}': debe ser de dos dígitos (00-23)"
    if len(min_str) != 2 or not es_solo_digitos(min_str):
        return f"Minutos inválidos '{min_str}': debe ser de dos dígitos (00-59)"

    hora = int(hora_str)
    minuto = int(min_str)

    if hora < 0 or hora > 23:
        return f"Hora inválida '{hora}': debe estar entre 00 y 23"
    if minuto < 0 or minuto > 59:
        return f"Minutos inválidos '{minuto}': debe estar entre 00 y 59"

    return None
```

Propósito:

Valida que una cadena con formato HH:MM represente una hora correcta en formato 24 horas (horas de 00 a 23, minutos de 00 a 59).

Implementación:

Similar a la validación de fechas: divide por ':', verifica que cada parte tenga dos dígitos y que los valores estén dentro de los rangos permitidos.

Uso:

Se llama desde `aceptar_token()` cuando el estado es HORA_MIN, antes de generar el token HORA.

2-Funciones auxiliares del analizador sintáctico (parser.py)

2.1. `_token_actual()`

```
def _token_actual():  
    return lista_tokens[pos] if pos < len(lista_tokens) else None
```

Propósito:

Retorna el token en la posición actual del flujo de tokens (o None si ya se alcanzó el final).

Abstrae el acceso a la lista global `lista_tokens` y la variable `pos`.

Uso:

Es utilizada por `consumir()` y `token_siguiente_es()` para consultar el token sin necesidad de repetir el mismo código.

2.2. `consumir(tipo_esperado, valor_esperado=None)`

```
def consumir(tipo_esperado, valor_esperado=None):  
    global pos  
    token_actual = _token_actual()  
  
    if token_actual is None:  
        ultimo = lista_tokens[-1] if lista_tokens else None  
        linea = ultimo.Linea if ultimo else None  
        columna = ultimo.columna if ultimo else None  
        descripcion_esperada = tipo_esperado  
        if valor_esperado is not None:  
            opciones = valor_esperado if isinstance(valor_esperado, (list, tuple)) else [valor_esperado]  
            descripcion_esperada += f" ({' o '.join(repr(o) for o in opciones)})"  
        raise ErrorSintactico(  
            f"se esperaba {descripcion_esperada}, pero el archivo terminó antes de tiempo.",  
            linea, columna, _pila_contexto,  
            sugerencia="Revisá que todo bloque WHEN / EVERY / IF tenga su 'end' correspondiente."  
        )  
    #sigue
```

Propósito:

Verifica que el token actual coincida con el tipo (y opcionalmente el valor) esperado según la gramática. Si coincide, avanza la posición (`pos`) y devuelve el token; en caso contrario, lanza un `ErrorSintactico` con información detallada de línea, columna y contexto.

Implementación:

- Obtiene el token actual mediante `_token_actual()`.

- Si no hay token, genera un error de "archivo terminó antes de tiempo".
- Compara el tipo y, si se especifica valor_esperado, verifica que el valor del token (en minúsculas) esté dentro de las opciones dadas (que pueden ser un string o una lista de strings).
- Si todo es correcto, incrementa pos y retorna el token.

Uso:

Es la función fundamental del parser; se invoca en cada regla de producción para consumir los terminales esperados. Centraliza el manejo de errores de token inesperado y proporciona sugerencias útiles.

2.3. `token_siguiente_es(tipo, valor=None)`

```
def token_siguiente_es(tipo, valor=None):
    """Lookahead sin consumir: permite decidir qué producción aplicar
    (ej. si sigue 'when', 'every', 'if' o un dispositivo)."""
    t = _token_actual()
    if t is None or t.tipo != tipo:
        return False
    if valor is not None:
        opciones = valor if isinstance(valor, (list, tuple)) else [valor]
        return t.valor.strip().lower() in [o.lower() for o in opciones]
    return True
```

Propósito:

Realiza un *lookahead* (anticipación) sin consumir el token. Permite decidir qué producción aplicar en función del próximo token. Retorna True si el token actual coincide con el tipo y valor especificados.

Uso:

Se emplea en las funciones de parseo para distinguir entre WHEN, EVERY, IF, o para saber si hay un ELSE en un condicional, o si estamos ante una asignación.

2.4. `@contextmanager contexto(nombre)`

```

@contextmanager
def contexto(nombre):
    """Registra 'nombre' mientras se ejecuta el bloque `with`, para que
    cualquier ErrorSintactico lanzado adentro incluya el camino completo
    de producciones (ej. PROGRAMA -> BLOQUE_WHEN -> LISTA_ACCIONES -> ASIGNACION)."""
    _pila_contexto.append(nombre)
    try:
        yield
    finally:
        _pila_contexto.pop()

```

Propósito:

Es un context manager (administrador de contexto) que apila el nombre de la producción sintáctica actual en la pila global `_pila_contexto`. Esto permite que, si se lanza un `ErrorSintactico` dentro del bloque, el error incluya la cadena completa de producciones anidadas (p. ej., PROGRAMA -> BLOQUE_WHEN -> LISTA_ACCIONES -> ASIGNACION), mejorando enormemente la información de depuración.

Implementación:

Al entrar al bloque, agrega nombre a la pila; al salir, lo remueve. Si ocurre una excepción, la pila se restaura gracias al bloque `finally`.

Uso:

Cada función de parseo que representa una producción no terminal se envuelve con `with contexto("NOMBRE");`, de modo que cualquier error que ocurra dentro quedará contextualizado.

2.5. `_error_atributo_no_manejado(dispositivo, atributo, token)`

```

def _error_atributo_no_manejado(dispositivo, atributo, token):
    """El lexer ya valida que el atributo exista para el dispositivo
    (ATRIBUTOS_POR_DISPOSITIVO), así que esta rama es defensiva: solo se
    activaría si se agrega un atributo nuevo al lexer sin actualizar acá."""
    validos = ATRIBUTOS_POR_DISPOSITIVO.get(dispositivo, [])
    raise ErrorSintactico(
        f"el atributo '{atributo}' no está contemplado en la gramática del parser para '{dispositivo}'.",
        token.linea, token.columna, _pila_contexto,
        sugerencia=f"Atributos válidos para {dispositivo}: {' '.join(validos)}."
    )

```

Propósito:

Función defensiva que se invoca cuando se detecta un atributo que, aunque el lexer lo considera válido para el dispositivo, no está contemplado en las reglas de producción del parser. Esto puede ocurrir si se modifica la tabla `ATRIBUTOS_POR_DISPOSITIVO` en el lexer sin actualizar el parser.

Implementación:

Construye un mensaje de error con sugerencia que lista los atributos válidos para ese dispositivo (obtenidos del lexer).

Uso:

Se utiliza en `_parse_expresion_dispositivo()` y en `parse_asignacion()` como fallback para atributos no esperados.

2.6. Validaciones semánticas específicas

Estas funciones refuerzan restricciones semánticas que el lexer no puede verificar por carecer de contexto sintáctico.

- `_consumir_color()`

```
def _consumir_color():
    token = consumir("VALOR_DISCRETO")
    if token.valor.strip().lower() not in VALORES_COLOR:
        raise ErrorSintactico(
            f'"{token.valor}" no es un color válido para .color.',
            token.linea, token.columna, _pila_contexto,
            sugerencia=f"Colores válidos: {' '.join(sorted(VALORES_COLOR))}."
        )
    return token
```

- Consume un token de tipo VALOR_DISCRETO y verifica que su valor esté dentro del conjunto {blanco, rojo, azul}. Si no, lanza un error con sugerencia.

- `_consumir_modos_aire()`

```
def _consumir_modos_aire():
    token = consumir("VALOR_DISCRETO")
    if token.valor.strip().lower() not in VALORES_MODALOS_AIRE:
        raise ErrorSintactico(
            f'"{token.valor}" no es un modo válido para .aire.modos.',
            token.linea, token.columna, _pila_contexto,
            sugerencia=f"Modos válidos: {' '.join(sorted(VALORES_MODALOS_AIRE))}."
        )
    return token
```

- Similar a la anterior, pero para el modo del aire acondicionado: valida que el valor sea {frio, calor, vent}.
- `_consumir_temperatura_rango(minimo, maximo, nombre_campo)`

```

def _consumir_temperatura_rango(minimo, maximo, nombre_campo):
    """El lexer solo valida el rango genérico (-10 a 50, el de sensor_temp /
    aire.temp_act). Algunos atributos, como aire.temp_obj, tienen un rango
    más estricto (16 a 30) que depende del contexto sintáctico, así que se
    revalida acá."""
    token = consumir("TEMPERATURA")
    texto_numero = token.valor.lower().replace('°', '').replace('c', '')
    try:
        num = int(texto_numero)
    except ValueError:
        # El lexer ya garantiza formato numérico válido; esto es defensivo.
        return token
    if not (minimo <= num <= maximo):
        raise ErrorSintactico(
            f"el valor de {nombre_campo} ({token.valor}) está fuera del rango permitido "
            f"({minimo}°C a {maximo}°C).",
            token.linea, token.columna, _pila_contexto,
            sugerencia="El lexer solo valida el rango general (-10 a 50 °C); "
        )
    return token

```

- Consume un token TEMPERATURA, extrae el valor numérico (eliminando °C) y verifica que esté dentro del rango [minimo, maximo]. Se utiliza para aire.temp_obj (rango 16-30°C), que es más restrictivo que el rango general del lexer (-10 a 50°C).

Propósito común:

Estas funciones aseguran que los valores asignados o comparados cumplan con la semántica del dispositivo/atributo, evitando errores lógicos que el lexer no podría detectar.

3. Clases auxiliares para el control de errores

3.1. ErrorLexico (en *lexer.py*)

Almacena un mensaje, línea y columna de un error léxico. Su representación en cadena (**repr**) formatea la información de manera legible. Es utilizada por el lexer para reportar errores sin interrumpir el análisis.

3.2. ErrorSintactico (en *parser.py*)

Extiende Exception y añade atributos para línea, columna, pila de contexto (contexto) y sugerencia (sugerencia). Su método `_formatear()` construye un mensaje completo y detallado que incluye la traza de producciones activas. Esto permite al usuario entender no solo dónde falló, sino también en qué construcción sintáctica estaba el parser.

4. Integración de las funciones auxiliares

Todas estas funciones colaboran para que los analizadores sean modulares, legibles y robustos:

- El lexer utiliza funciones de validación para asegurar que los literales (fechas, horas, porcentajes, etc.) cumplan con los rangos y formatos especificados.
- El parser emplea `consumir()` y `token_siguiente_es()` para navegar por el flujo de tokens de manera segura, mientras que el contexto y las validaciones semánticas garantizan que la estructura y los valores sean correctos.
- Las clases de error permiten una retroalimentación clara y precisa, facilitando la depuración de scripts de usuario.

Modo de Obtención del Intérprete

Para obtener y preparar el entorno de ejecución del intérprete del lenguaje SMART HOME, el usuario debe seguir el siguiente procedimiento técnico:

1. Entorno y Requisitos Mínimos

El sistema está desarrollado íntegramente en Python 3. Al apoyarse exclusivamente en módulos nativos de la biblioteca estándar de Python (como `sys` y `os`), no se requiere la instalación de dependencias de terceros (`pip`, `virtualenv`). Esto garantiza la portabilidad inmediata del software.

2. Estructura de Archivos Necesarios

Para que el intérprete funcione correctamente, la carpeta raíz del código fuente (`src/`) debe contener de forma obligatoria los siguientes módulos interconectados:

- `main.py`: Punto de entrada principal y gestor de la interfaz de usuario.
- `lexer.py`: Motor del analizador léxico (Autómata Finito Determinístico).
- `parser.py`: Analizador sintáctico descendente recursivo y traductor.

3. Descarga desde el Repositorio

El código fuente optimizado se encuentra alojado en la plataforma GitHub, la cual fue adoptada en nuestra alianza grupal para asegurar el control de versiones asincrónicas mediante Git. El software puede obtenerse clonando el repositorio mediante la consola de comandos o descargando directamente el paquete comprimido en formato `.zip` desde la interfaz web de la plataforma, procediendo luego a su extracción en un directorio local con permisos de lectura y escritura.

<https://github.com/julianutnresistencia-del/MULTIVAC>

Modo de Ejecución del Intérprete

1. Modo Interactivo (Consola REPL)

Diseñado para realizar pruebas rápidas de fragmentos de código. Se activa al ejecutar el archivo `main.exe` ubicado en la carpeta `bin`

- **Funcionamiento:** El sistema inicia una sesión interactiva en bucle que solicita cadenas de texto al usuario de forma manual. Cada instrucción ingresada es procesada inmediatamente por el lexer y, si no presenta anomalías léxicas, es evaluada por el parser. El bucle ofrece un control de salida amigable: finaliza si se ingresa la palabra clave `salir` o si el usuario responde con un 0 ante la consulta de continuidad. En caso de procesarse bloques válidos, los resultados se guardan por defecto en el archivo `main.html`.

2. Modo Archivo (Procesamiento por Lotes)

Es la modalidad principal del intérprete, ideal para ejecutar programas complejos de automatización residencial multilínea de forma estructurada.

- Se activa al ejecutar el archivo `gui.exe` ubicado en la carpeta `bin`
- **Controles de ejecución integrados:** Antes de iniciar el procesamiento del flujo de caracteres, el punto de entrada implementa una serie de capas de software defensivo que validan el entorno del archivo físico:
 1. **Validación de extensión:** Comprueba mediante `os.path.splitext` que el archivo de entrada posea estrictamente la extensión `.smart`. Si no la tiene (o carece de extensión), interrumpe informando el error.
 2. **Validación de existencia:** Verifica que la ruta provista exista verdaderamente en el disco rígido.
 3. **Validación de tipo:** Constata que el elemento apuntado sea un archivo regular ejecutable y no un directorio.
 4. **Control de lectura seguro:** Envuelve la apertura del archivo en un bloque `try/except` para capturar de manera segura excepciones de sistema (`OSError`) o problemas de codificación (`UnicodeDecodeError`) al leer el archivo en formato UTF-8.
- **Generación automática de salida:** Al procesar un script en este modo, el sistema analiza el contenido completo de manera multilínea para preservar la consistencia de filas y columnas. Si el programa no presenta fallas, el parser extrae las colecciones de datos, genera las estructuras correspondientes y escribe de manera automática un archivo de salida `.html`

que hereda el mismo nombre base del script original (por ejemplo, si se procesa casa.smart, se creará de forma transparente el archivo casa.html con las tablas y bloques CSS de sensores y actuadores).

Conclusiones

Como cierre del trabajo presentado en el informe, se concluye que el desarrollo de este Trabajo Práctico Integrador para la cátedra de Sintaxis y Semántica de los Lenguajes de la permitió al grupo MULTIVAC consolidar con éxito los conceptos teóricos de autómatas y gramáticas mediante el diseño e implementación del lenguaje SMART HOME. La decisión de construir un lexer basado en un AFD programado manualmente, junto con un parser recursivo descendente, garantizó un análisis preciso, independiente de la configuración regional del sistema y capaz de resolver ambigüedades. Asimismo, la integración de un sistema defensivo de control de errores apoyado en un administrador de contexto optimiza notablemente la depuración para el usuario, mientras que la arquitectura modular del software asegura la facilidad de futuras expansiones del lenguaje. Finalmente, la coordinación grupal estructurada a través de GitHub fue clave para potenciar las habilidades individuales de cada integrante y lograr un flujo de desarrollo ordenado, demostrando la viabilidad técnica de este intérprete y traductor a HTML.

Anexo

Presentacion:

<https://gamma.app/docs/Sintaxis-y-Semantica-de-los-Lenguajes-jsyp3hyiw3w4fb>

Video Informativo:

<https://youtu.be/78ucFI0YTqU>

Ejemplo de Árbol

